

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/ex03.git
cd ex03
```

2. 파일 구조

ex03 디렉토리

```
ex03/
├─ order/           # 포트 8081
├─ product/        # 포트 8082
├─ user/           # 포트 8083
├─ delivery/       # 포트 8084
├─ orchestrator/   # Kafka 워크플로우 조울 (이번 챕터 신규)
├─ gateway/        # Nginx API Gateway
├─ db/             # MySQL
└─ k8s/            # Kubernetes 매니페스트 (kafka 포함)
```

서비스마다 패키지 구조가 조금씩 다르므로, 코드를 작성할 파일 경로는 각 실습 코드블록 바로 위에서 안내합니다.

3. Kafka

이 챕터는 Kafka를 별도로 설치하지 않습니다. Kubernetes 매니페스트(`k8s/kafka/`)로 Minikube 안에 띄우므로, 챕터 3에서 준비한 Docker Desktop과 Minikube만 있으면 됩니다.

4. 실습 순서

1. order-service의 REST 호출을 Kafka 이벤트 발행으로 교체
2. product-service · delivery-service에 Kafka Consumer/Producer 추가
3. 새 서비스 **orchestrator**에서 워크플로우 조울 로직 작성
4. K8s에 Kafka + orchestrator 배포
5. 정상 주문 / 품질 롤백 시나리오 검증

이번 챕터는 책에서 가장 복잡한 내용을 다룹니다. 여러 서비스가 동시에 Kafka 메시지를 주고받으며 상태가 바뀝니다. 전체 토픽맵(4.2.2)과 흐름 표(4.3.1~4.3.2)를 먼저 훑어보고 시작하면 길을 잃지 않습니다.

4.1 동기 호출의 한계 - 한 서비스가 멈추면 전체가 멈춘다

배포 다음 날 아침, 모니터에 알림이 울렸습니다. 에러 그래프가 한 칸 솟았다가 곧 가라앉았습니다. 상품 서버가 잠깐 죽었고, 쿠버네티스가 다시 띄웠습니다.

그런데 그 잠깐 사이에 들어온 주문이 전부 실패해 있었습니다. 주문 서버는 재고를 줄이려고 상품 서버를 직접 호출했는데, 상품 서버가 죽어 있어 호출이 실패했습니다. 주문 처리는 그 응답을 기다리던 중이라, 호출이 실패하면서 주문 처리도 함께 실패했습니다.

서로 직접 부르니까, 하나가 멈추면 기다리던 쪽도 같이 멈추는구나.

선배가 모니터를 들여다봤습니다.

선배 : "동기로 직접 호출하니까 그래요. 호출한 쪽은 상대가 응답할 때까지 기다리는데, 상대가 잠깐 죽으면 호출한 쪽도 같이 멈춰요. 이럴 땐 호출을 붙잡고 기다리지 말고 비동기로 메시지를 주고받아야 해요. 그 비동기 통신을 해 주는 게 **Kafka**예요."

오픈이 : "메시지요? 상대가 없으면 어떻게 되는데요?"

선배 : "주문하는 쪽은 메시지를 남기고 진동벨을 받고 돌아가요. 재고 담당자는 자기 차례가 되면 쌓인 메시지를 꺼내 처리해요. 담당자가 잠깐 자리를 비워도 메시지는 그 자리에 남아 있어요. 돌아와서 그때 처리하면 되죠."

선배 : "결국 이게 MSA가 노린 거예요. 서비스를 따로 배포하고, 하나가 죽어도 나머지는 멀쩡해야죠. 그런데 서로 직접 호출하면서 응답을 기다리는 한, 하나가 멈추면 다 같이 멈춰요. 카페로 치면 카운터 앞에서 기다리는 것과 진동벨을 받고 자리에 앉는 것의 차이예요. 그게 바로 동기와 비동기의 차이예요."

4.1.1 카운터 대기 vs 진동벨

카운터 대기 방식 (동기적)

카운터 앞에 서서 "아메리카노 하나요"라고 주문합니다. 바리스타가 커피를 만드는 동안 카운터 앞에서 기다립니다. 내 커피가 나와야 다음 손님이 주문할 수 있습니다. 바리스타가 원두를 갈다가 커피 머신이 고장 나면 뒤에 줄 선 손님 전부가 멈춥니다.

진동벨 방식 (비동기적)

"아메리카노 하나요"라고 주문하면 진동벨을 받고 자리에 앉습니다. 바리스타는 주문을 순서대로 처리하고, 커피가 완성되면 벨이 울립니다. 손님은 기다리는 동안 다른 일을 할 수 있고, 뒤에 줄 선 손님도 바로 주문할 수 있습니다. 커피 머신이 잠깐 멈춰도 주문 목록은 사라지지 않습니다.

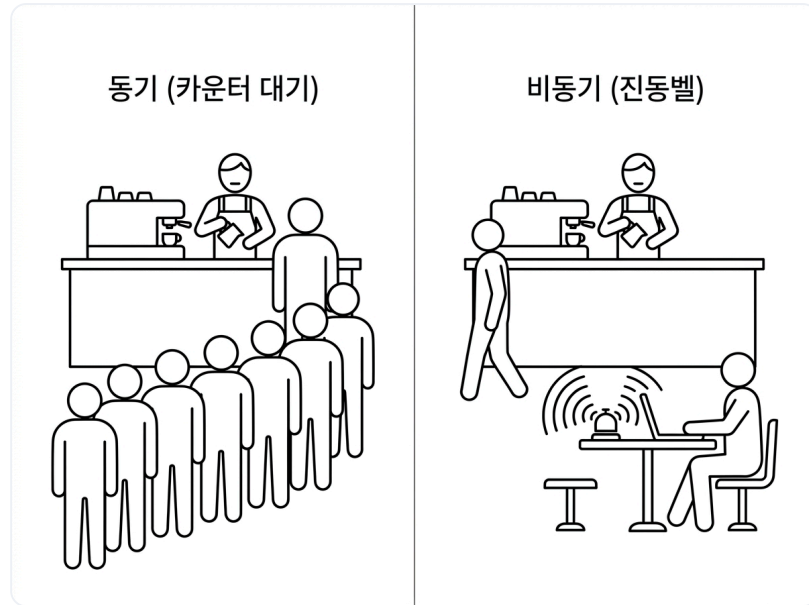


그림 4-2. 동기 vs 비동기 통신

아침에 겪은 그 잠깐의 멈춤이 바로 카운터 대기의 풍경이었습니다. 커피 머신이 멈추자 카운터 앞에 선 손님까지 함께 멈췄고, 그 사이 들어온 주문이 전부 실패했습니다. 진동벨 방식이라면 손님은 주문만 남기고 자리로 돌아가 있으니, 커피 머신이 잠깐 멈춰도 주문은 그대로 남아 있다가 커피 머신이 복구된 후에 처리됩니다. 동기로 서로를 직접 호출하며 응답을 기다리면 한 서비스의 멈춤이 곧 전체의 멈춤이 되지만, 비동기로 메시지를 주고받으면 그렇지 않습니다.

4.2 Kafka - 메시지를 전달하는 우체국

비동기 통신을 실제로 담당하는 것이 **Kafka**입니다.

Apache Kafka는 서비스끼리 직접 호출하지 않고, 가운데에서 메시지를 받아 보관했다가 전달해주는 메시지 시스템입니다. 받는 쪽이 잠시 멈춰도 메시지가 그대로 남아 있어 비동기 통신이 가능합니다.

Kafka는 우체국입니다. 발신자가 우편을 부치면, 우체국은 **일반 편지, 등기, 특송, 택배처럼 종류별로 나눠** 따로 보관합니다. 그리고 집배원은 자기가 처리할 종류의 칸에서만 우편을 꺼내 갑니다. 우체국을 사이에 두고 우편을 주고받기 때문에, 비동기적으로 각자의 시간에 일을 처리할 수 있습니다.



그림 4-3. Kafka 우체국 - 부치고, 보관·분류되고, 집배원이 가져간다

그림 4-3의 Kafka 요소의 역할을 정리하면 다음과 같습니다.

구성요소	우체국 비유	하는 일
Kafka	우체국	메시지를 받아 보관하고 전달
토픽(Topic)	종류별 우편함 (일반/등기/특송 등)	메시지를 목적별로 나눠 담음
프로듀서(Producer)	발신자	토픽에 메시지를 발행
컨슈머(Consumer)	집배원	토픽을 구독해 메시지를 처리

프로듀서가 토픽에 메시지를 보내는 일을 **발행**, **컨슈머**가 자기가 다룰 토픽을 정해 두고 거기에 들어온 메시지를 받아 처리하는 일을 **구독**이라고 합니다.

그런데 같은 일을 하는 컨슈머가 여러 대 떠 있을 때는 메시지를 어떻게 나눠 받아야 할까요?

4.2.1 컨슈머 그룹

상품 서비스를 2대로 늘려 운영한다고 가정해 보겠습니다. 두 서버가 같은 토픽을 구독하면 "재고 1개 차감" 메시지를 둘 다 받아 처리해, 재고가 2개 줄어듭니다. 이 중복 처리를 막아 주는 것이 **컨슈머 그룹**입니다. 같은 일을 하는 서버를 한 그룹으로 묶으면, 그 메시지는 그룹 안의 한 서버에만 전달됩니다.

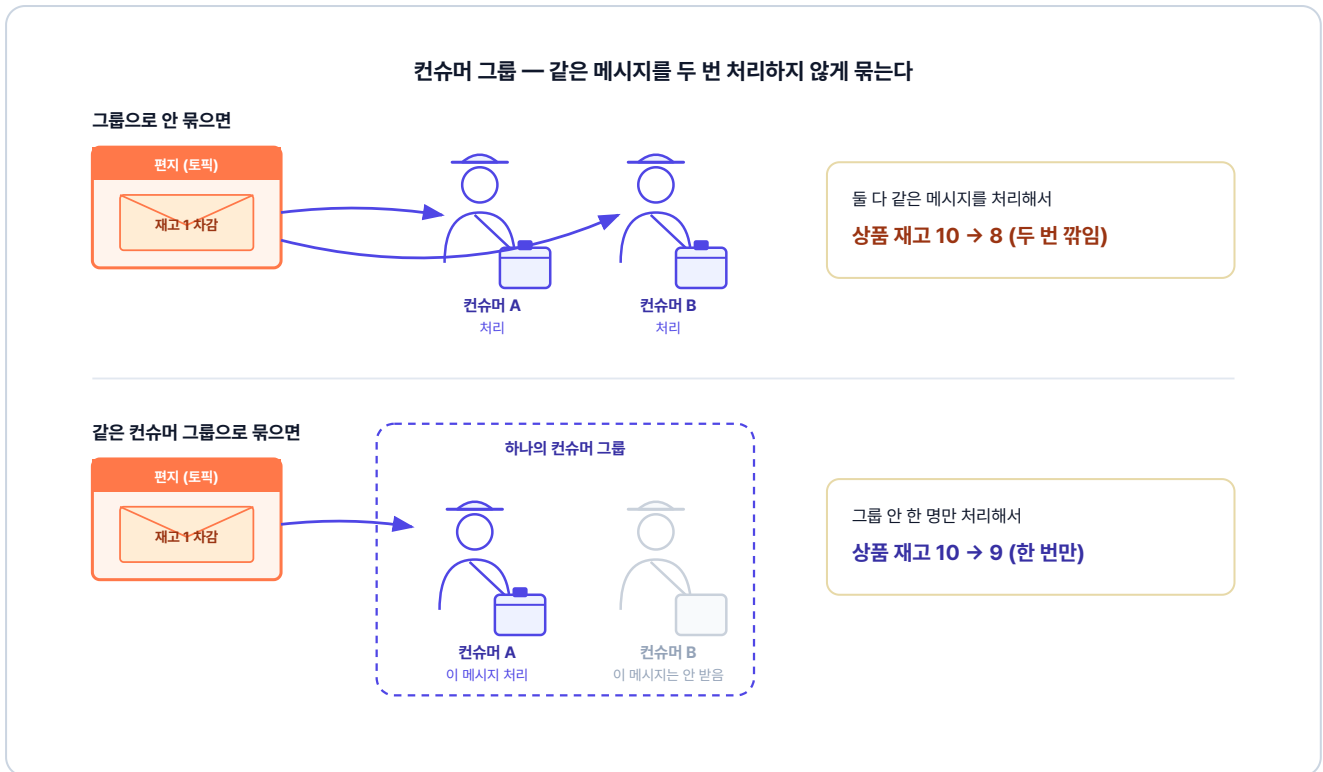


그림 4-4. 컨슈머 그룹 - 안 묶으면 재고가 두 번 깎이고, 같은 그룹으로 묶으면 한 번만 깎인다

4.2.2 이번 챕터에서 사용하는 토픽 맵

이 챕터에서는 핵심 흐름만 직접 다루므로, 각 토픽의 발행·구독 코드 전체는 깃헙 레포에서 확인합니다.

총 8개의 토픽을 사용합니다. `command` 가 붙은 토픽은 orchestrator가 각 서비스에 내리는 명령입니다.

`command` 가 없는 토픽은 각 서비스가 처리 결과를 orchestrator에 보고하는 이벤트입니다.

정상 흐름 (6단계)

단계	토픽	발행 → 구독	목적
1	<code>order-created</code>	order → orchestrator	새 주문 발생
2	<code>decrease-product-command</code>	orchestrator → product	재고 감소 명령
3	<code>product-decreased</code>	product → orchestrator	재고 감소 결과
4	<code>create-delivery-command</code>	orchestrator → delivery	배달 생성 명령
5	<code>delivery-created</code>	delivery → orchestrator	배달 생성 결과
6	<code>complete-order-command</code>	orchestrator → order	주문 완료 명령

롤백 흐름 (2개)

토픽	발행 → 구독	목적
<code>cancel-order-command</code>	orchestrator → order	주문 취소
<code>increase-product-command</code>	orchestrator → product	재고 복구

오픈이 : "메시지 방식으로 바꾸니까, 전체 흐름을 누가 관리해야 할지 모르겠어요."

선배 : "전체 흐름을 관리하는 **지휘자**를 만들면 편해요. 전체 악보를 보고 순서를 정하는 것처럼, 각 서비스는 자기 일만 하고 결과만 보고하게 해요."

Kafka 더 알아보기

- **컨슈머가 읽어도 메시지는 남는다**: 우체국에서 우편을 꺼내 읽어도 그 우편은 함에 그대로 보관됩니다(기본 7일). 그래서 서로 다른 컨슈머가 같은 메시지를 각자 자기 속도로 찾아 읽을 수 있고, 장애가 나도 다시 읽어 처리할 수 있습니다.
- **RabbitMQ 같은 큐와의 차이**: 전통적인 메시지 큐는 컨슈머가 메시지를 읽으면 큐에서 바로 사라집니다. Kafka는 읽어도 남아 있다는 점이 가장 큰 차이입니다.

4.3 Orchestration Saga - 지휘자가 흐름을 조율하다

각 서비스가 자기 단계만 처리해 결과를 메시지로 발행하고, 별도의 관리자가 다음 단계와 보상을 책임지는 분산 트랜잭션 패턴을 **Saga 패턴**이라고 합니다.

Saga 패턴의 흐름 관리를 한 서비스가 중앙에서 지휘하는 방식이 **Orchestration Saga**, 그 지휘자 서비스가 **orchestrator**입니다. 흐름을 한 곳에 모으면 상태가 한 곳에 있어 추적과 롤백이 단순해지고, 각 서비스는 자기 일에만 집중할 수 있습니다.

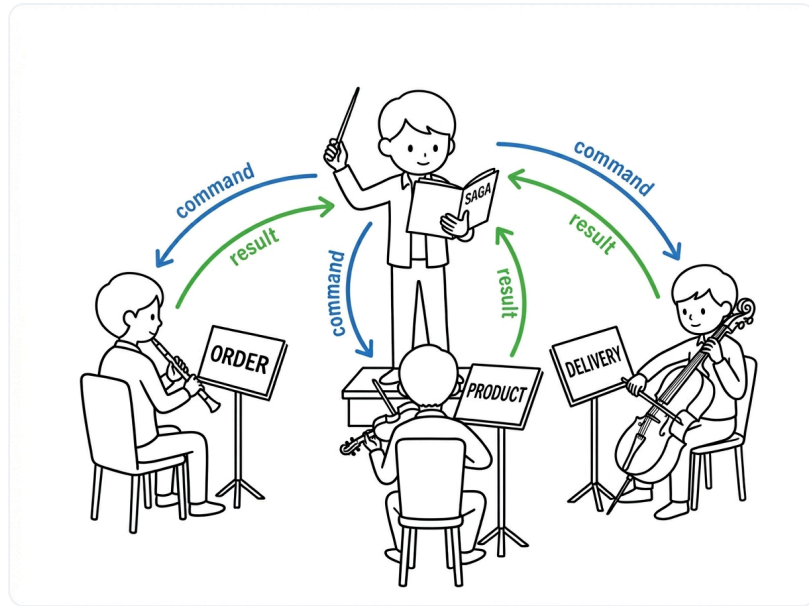


그림 4-5. Orchestration Saga 구조

이번 챕터의 예제에서는 주문 요청이 들어오면 orchestrator가 재고 차감, 배달 생성, 주문 완료 단계를 순서대로 지휘합니다.

Saga 패턴을 구현하는 방식에는 Choreography도 있습니다. 중앙 지휘자 없이 각 서비스가 이벤트를 발행하고 구독하며 다음 단계를 스스로 이어 갑니다. 단계가 적을 때는 가볍지만, 단계가 늘거나 보상이 복잡해질수록 전체 흐름이 여러 서비스에 흩어져 추적이 어렵습니다. 이 책은 상태를 한 곳에서 추적할 수 있는 Orchestration을 선택합니다.

4.3.1 주문 요청 성공 흐름

orchestrator를 사이에 두고 비동기로 동작하면, 주문 요청이 들어왔을 때 주문 서비스는 우선 **PENDING** 상태를 응답합니다. 이후 orchestrator가 아래 단계를 차례로 진행해 모두 성공하면 주문 상태가 **COMPLETED**로 바뀝니다.

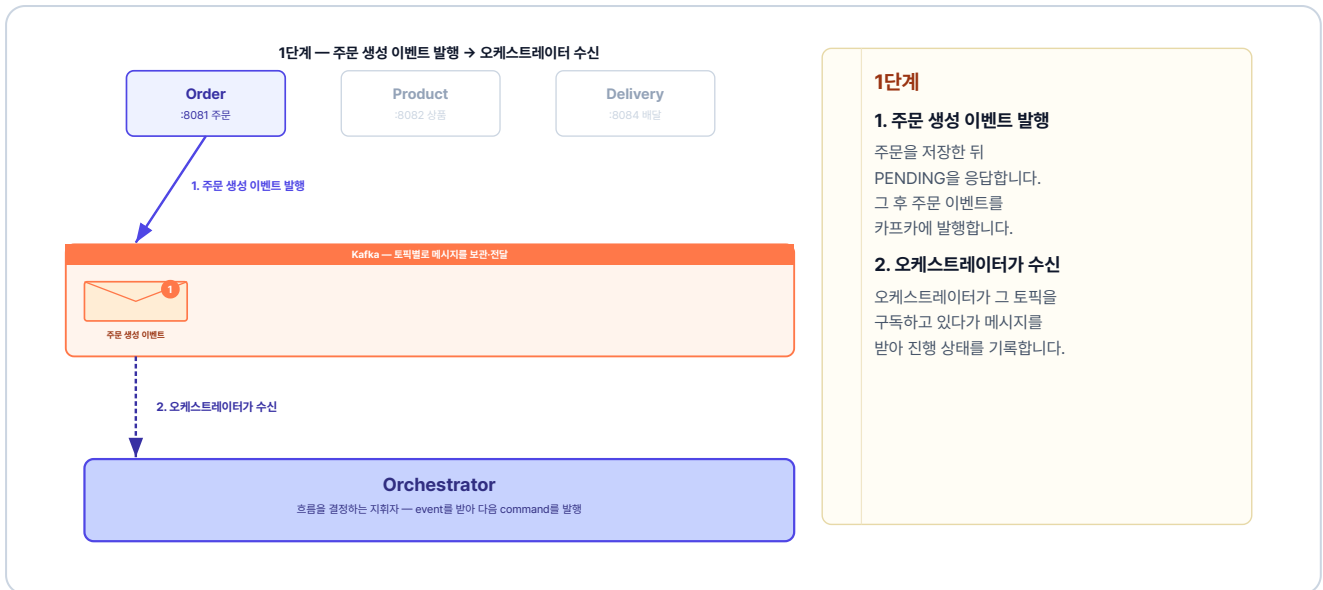


그림 4-6. 1단계 — 주문 생성 이벤트 발행 → 오케스트레이터 수신

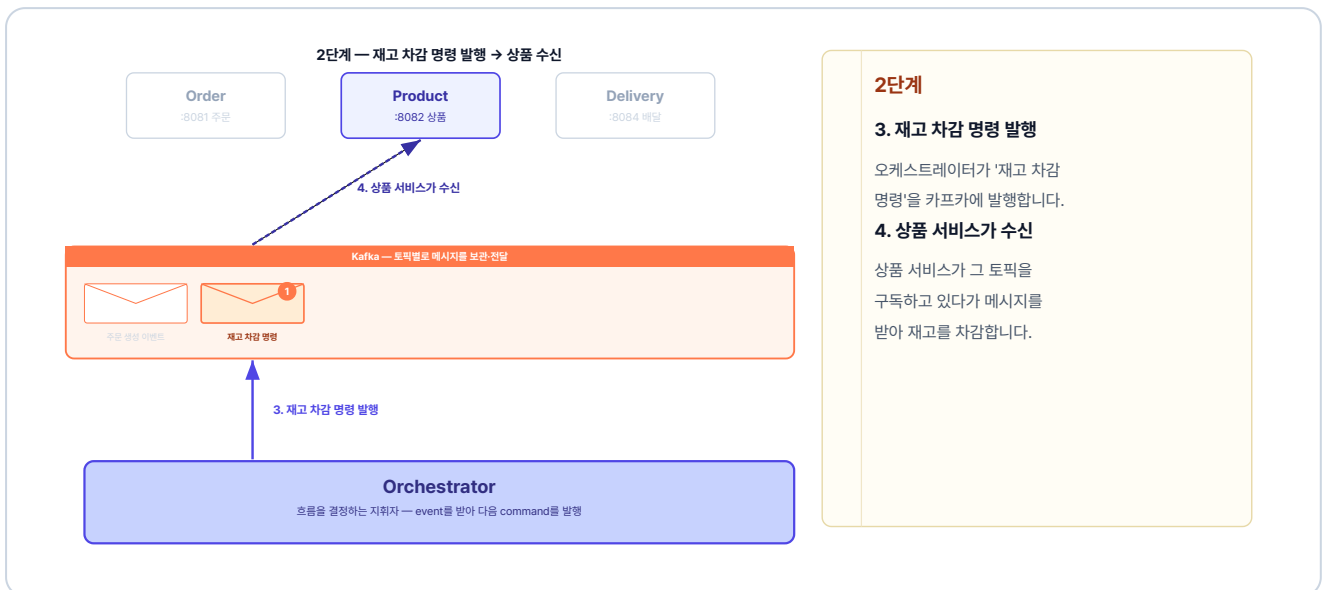


그림 4-7. 2단계 — 재고 차감 명령 발행 → 상품 수신

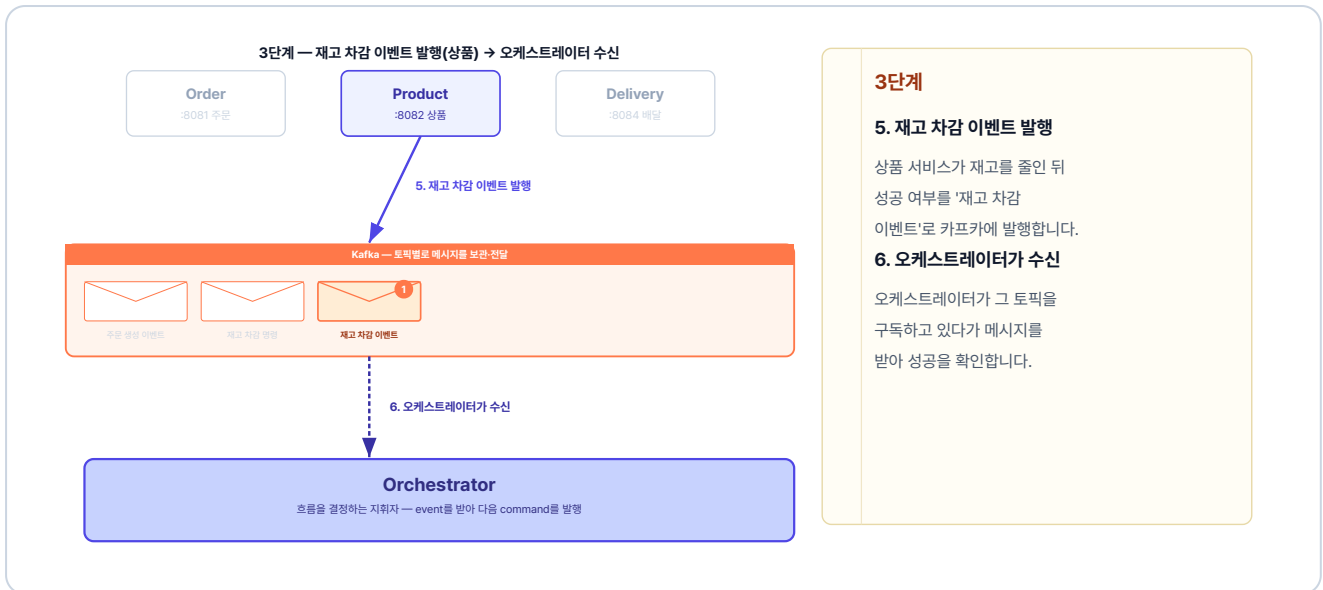


그림 4-8. 3단계 — 재고 차감 이벤트 발행(상품) → 오케스트레이터 수신

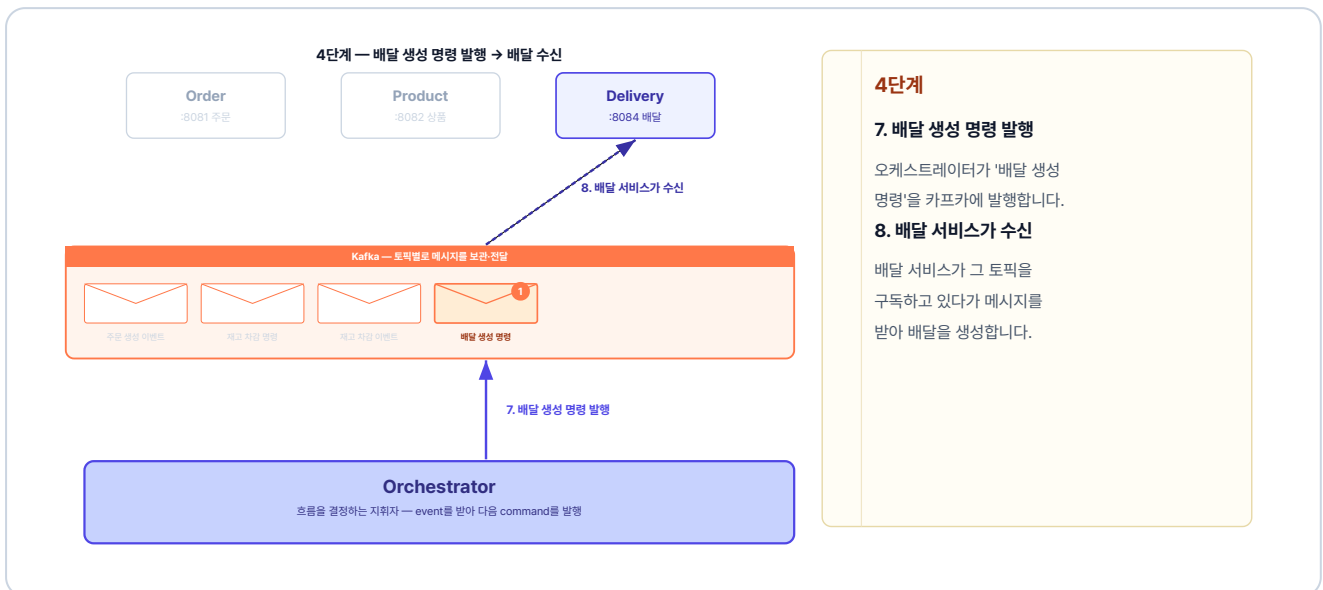


그림 4-9. 4단계 — 배달 생성 명령 발행 → 배달 수신

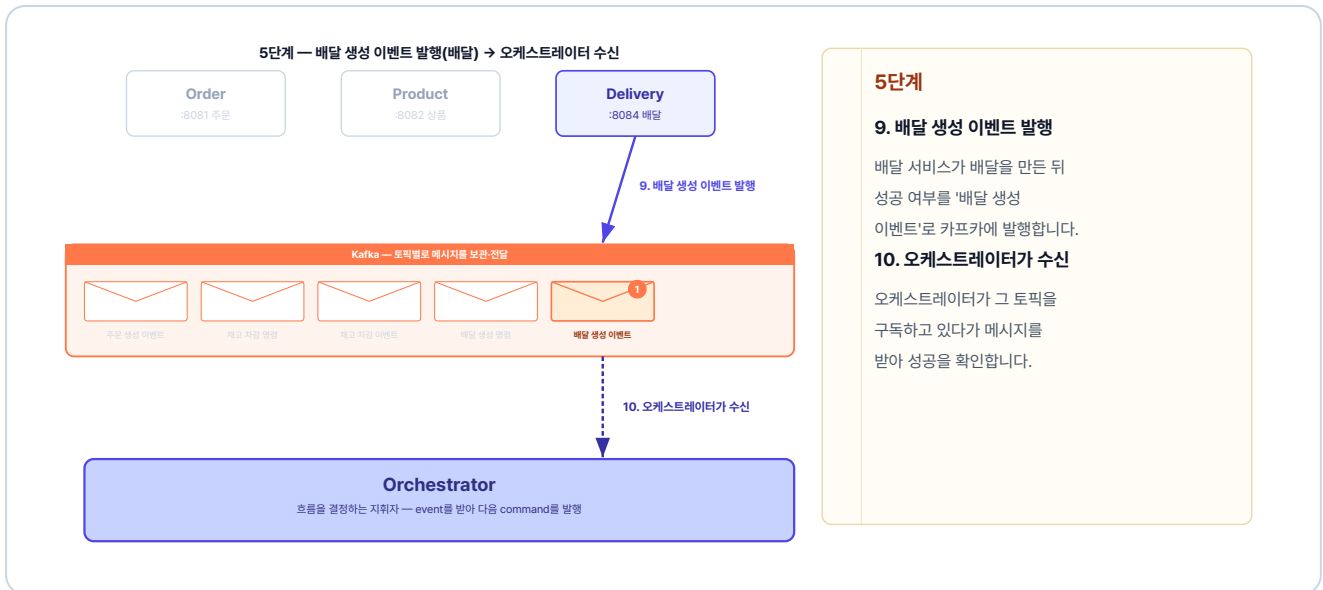


그림 4-10. 5단계 — 배달 생성 이벤트 발행(배달) → 오케스트레이터 수신

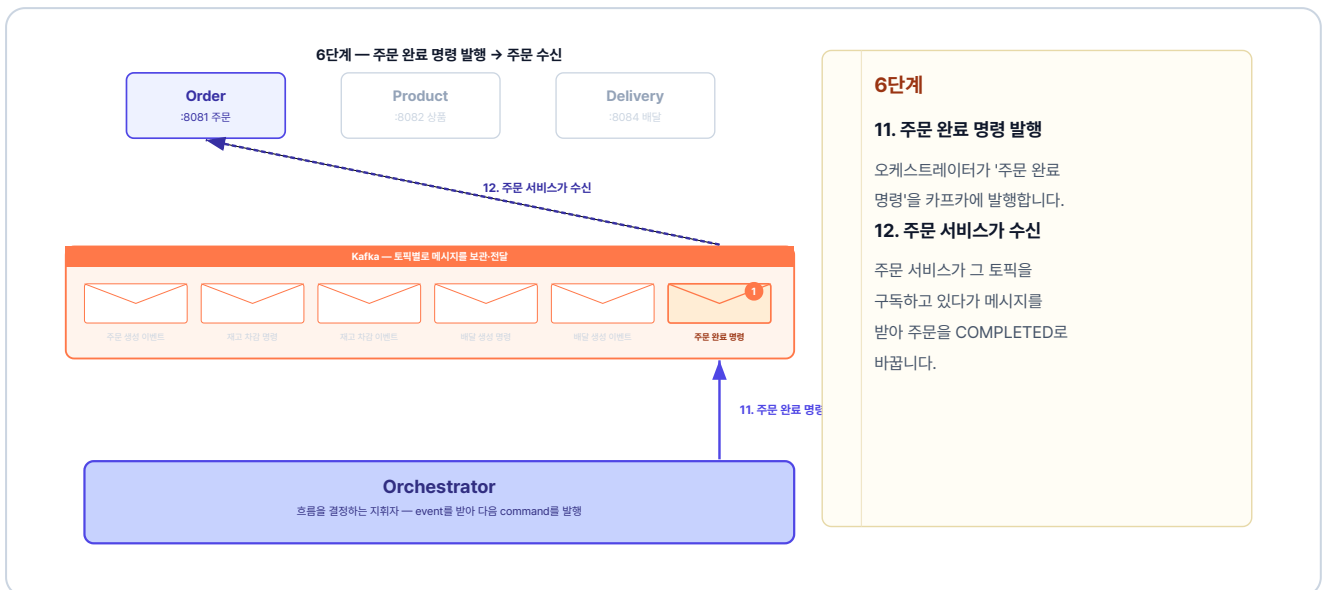


그림 4-11. 6단계 — 주문 완료 명령 발행 → 주문 수신

4.3.2 주문 요청 실패 흐름 (롤백)

상품 서비스가 재고 차감에 실패하면 재고 차감 실패 이벤트를 발행합니다. 오케스트레이터는 이미 처리된 단계만 역순으로 되돌립니다.



그림 4-12. 롤백 · 실패 알림 — 재고 차감에 실패한 상품이 그 결과를 오케스트레이터에 알립니다

되돌릴 재고가 없으니 남은 일은 주문을 무르는 것입니다. 오케스트레이터는 주문 취소 명령을 발행하고, 주문 상태가 CANCELLED로 변경됩니다.

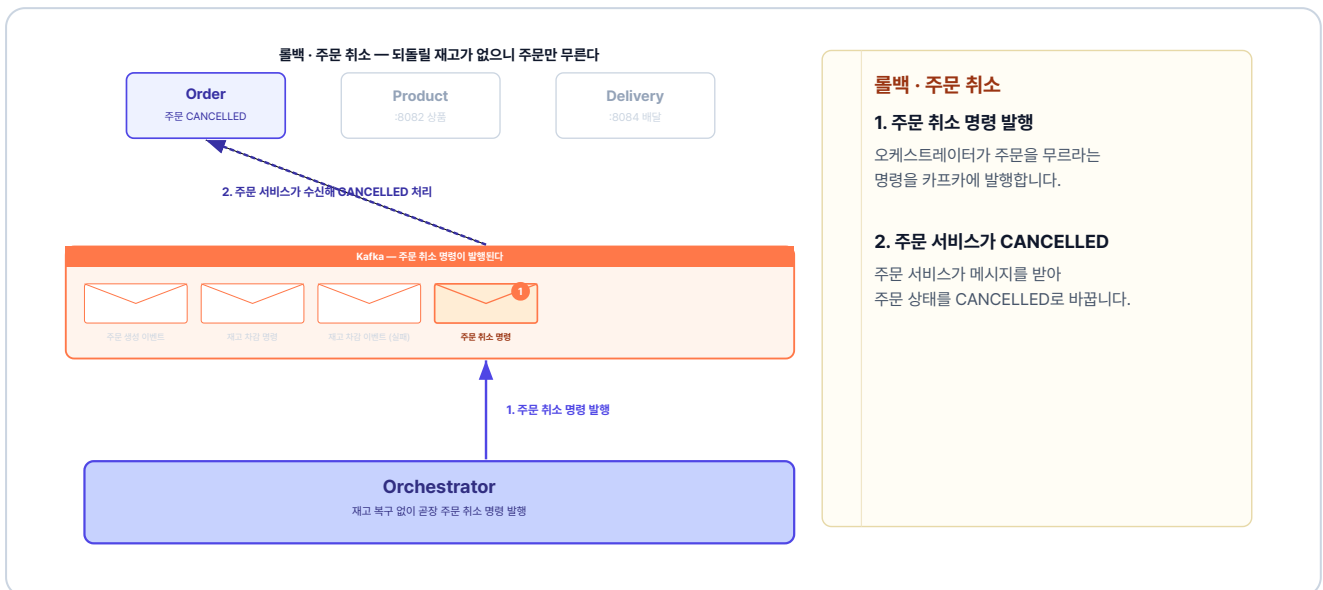


그림 4-13. 롤백 · 주문 취소 — 되돌릴 재고가 없으니, 주문 취소 한 단계로 롤백이 끝납니다

선배 : "흐름은 잡혔으니, 이제 직접 만들어 봐요."

4.4 Kafka로 주고받기 - 발행과 구독

4.4부터는 이 흐름을 코드로 옮깁니다. Kafka 코드는 메시지를 토픽에 넣는 발행과, 토픽을 듣고 있다가 받는 구독으로 이뤄집니다.

4.4.1 발행 - 토픽에 메시지를 넣는다

앞에서 본 발신자가 우편을 부치듯, 프로듀서도 메시지를 토픽에 부칠 도구가 필요합니다. Spring에서 그 도구로 제공하는 것이 `KafkaTemplate` 입니다. `"order-created"` 처럼 이름을 토픽 자리에 넘기면 그 토픽으로 메시지가 들어갑니다. 주문 서비스가 주문을 저장한 뒤 '주문 생성 이벤트'를 발행하는 코드는 다음과 같습니다.

`adapter/producer/OrderEventProducer.java` 를 열고 아래 메서드를 작성합니다.

실습 1 `adapter/producer/OrderEventProducer.java`. 주문 생성 이벤트 발행

```
@Component
@RequiredArgsConstructor
public class OrderEventProducer {
    private final KafkaTemplate<String, Object> kafkaTemplate;

    public void publishOrderCreated(OrderCreatedEvent event) {
        // "order-created" 토픽에 이벤트를 넣는다
        kafkaTemplate.send("order-created", event);
    }
}
```

`kafkaTemplate.send` 에 토픽 이름과 보낼 객체를 넘기면 해당 토픽으로 메시지가 들어갑니다.

주문 서비스는 주문을 저장하고 '주문 생성 이벤트'를 발행하면 됩니다. 그다음 재고 차감·배달 생성·실패 복구는 orchestrator가 맡습니다.

4.4.2 orchestrator - 흐름을 조율하는 코드

이벤트를 받아 다음 명령을 정하는 일은 orchestrator만 합니다. 1단계(주문 생성 이벤트 수신)를 코드로 보면 다음과 같습니다.

`@KafkaListener`의 `topics`는 구독할 토픽 이름입니다. 토픽은 메시지가 종류별로 쌓이는 우편함이고, "order-created"처럼 그 우편함 이름을 적습니다. `groupId`는 앞에서 본 컨슈머 그룹의 이름입니다. 같은 `groupId`를 적은 컨슈머끼리 한 그룹이 됩니다.

`handler/OrderOrchestrator.java`를 열고 아래 메서드를 작성합니다.

실습 2 `handler/OrderOrchestrator.java`. 주문 생성 이벤트를 받아 재고 차감 명령 발행

```
@KafkaListener(topics = "order-created", groupId = "orchestrator")
public void orderCreated(OrderCreatedEvent event) {
    // 1. 주문 진행 상태를 메모리에 기록
    states.put(event.orderId(), new WorkflowState(
        event.orderId(), event.address(),
        event.productId(), event.quantity(), event.price()));

    // 2. 다음 단계: 재고 차감 명령 발행
    kafkaTemplate.send("decrease-product-command",
        new DecreaseProductCommand(event.orderId(),
            event.productId(), event.quantity(), event.price()));
}
```

이 메서드는 주문 생성 이벤트를 받아 진행 상태를 기록하고, 다음 명령을 발행합니다. 그림 4-6·4-7의 흐름이 이 한 메서드에 그대로 담겨 있습니다.

`WorkflowState`는 주문 한 건의 진행 정보를 메모리에 들고 있는 객체입니다. 결과 이벤트가 돌아올 때 orchestrator는 이 기록을 보고 다음 명령을 정합니다.

4.4.3 구독 - 토픽의 메시지를 받는다

앞에서 orchestrator가 발행한 '재고 차감 명령'을 이번에는 상품 서비스가 받습니다. 받는 쪽은 메서드에 `@KafkaListener`를 붙이고 토픽 이름과 그룹 이름을 적습니다. 그 토픽에 메시지가 들어오면 이 메서드가 자동으로 실행됩니다. 상품 서비스는 이 명령을 받아 재고를 줄이고, 그 결과를 '재고 차감 이벤트'로 발행합니다.

`adapter/consumer/ProductCommandConsumer.java`를 열고 아래 메서드를 작성합니다.

실습 3 adapter/consumer/ProductCommandConsumer.java. 재고 차감 명령 구독

```
@KafkaListener(topics = "decrease-product-command", groupId = "product-service")
public void decreaseProductCommand(DecreaseProductCommand command) {
    boolean success = false;
    // 1. 재고 차감 (성공하면 success = true)
    try {
        productService.decreaseQuantity(command.productId(), command.quantity(), command.price());
        success = true;
    } catch (Exception e) {
        // 재고 부족 등 실패는 success = false로 그대로 보고
    }

    // 2. 처리 결과를 '재고 차감 이벤트'로 발행
    productEventProducer.publishProductDecreased(
        new ProductDecreasedEvent(command.orderId(), command.productId(), command.quantity(), success));
}
```

상품 서비스는 명령을 받아 재고를 줄이고, 성공·실패를 **success** 값에 담아 결과 이벤트로 돌려줍니다.

각 서비스 코드는 모두 같은 발행·구독 패턴이고, 토픽 이름만 다릅니다. 그래서 코드를 일일이 보지 않아도, 어떤 서비스가 무슨 토픽을 구독해 무엇을 하고 무엇을 발행하는지만 알면 전체가 보입니다.

서비스	구독(받음)	처리	발행(보냄)
order	주문 요청	주문 저장(PENDING)	주문 생성 이벤트
order	주문 완료·취소 명령	주문 상태 변경	—
product	재고 차감·복구 명령	재고 증감	재고 차감 이벤트
delivery	배달 생성 명령	배달 생성	배달 생성 이벤트
orchestrator	주문 생성·재고 차감·배달 생성 이벤트	다음 단계 결정	재고 차감·배달 생성·주문 완료 명령 (실패 시 복구·취소 명령)

각 서비스의 전체 코드는 GitHub에서 확인하세요.

이제 Kubernetes에 Kafka와 orchestrator를 추가하고 전체 시스템을 배포합니다.

4.5 Kubernetes - Kafka와 orchestrator 배포

챕터 3에서 6개 서비스를 Kubernetes에 올렸습니다. Kafka를 도입하면서 더해지는 건 둘입니다. Kafka 서버 한 대와 orchestrator 한 개. 그리고 기존 order-product-delivery는 Kafka가 어디 있는지 알아야 하므로 ConfigMap에 주소 한 줄을 더합니다. 챕터 3 배포에서 바뀌는 부분은 이 세 가지가 전부입니다.

K8s 리소스	역할
<code>kafka-deploy.yaml</code>	KRaft 모드로 Kafka 서버 한 대를 띄웁니다.
<code>kafka-service.yaml</code>	Kafka 서버를 <code>kafka-service:9092</code> 로 노출합니다.
<code>orchestrator-deploy.yaml</code>	워크플로우를 조율하는 orchestrator Pod를 띄웁니다. REST가 없어 Service는 두지 않습니다.
<code>orchestrator-configmap.yaml</code>	orchestrator에 환경변수를 주입합니다.

KRaft 모드는 Kafka가 클러스터 관리 정보를 스스로 처리하는 실행 방식입니다. 덕분에 별도 컴포넌트 없이 Kafka 컨테이너 하나로 동작합니다.

각 서비스는 Kafka 서버에 `kafka-service:9092` 주소로 접근합니다. 클라이언트 쪽은 각 서비스의 ConfigMap에 `SPRING_KAFKA_BOOTSTRAP_SERVERS`로 접속 주소를 적고, Kafka 서버 쪽은 `kafka-deploy.yaml`에 `KAFKA_ADVERTISED_LISTENERS`로 자기 주소를 알립니다. 둘 다 `kafka-service:9092`로 맞춥니다.

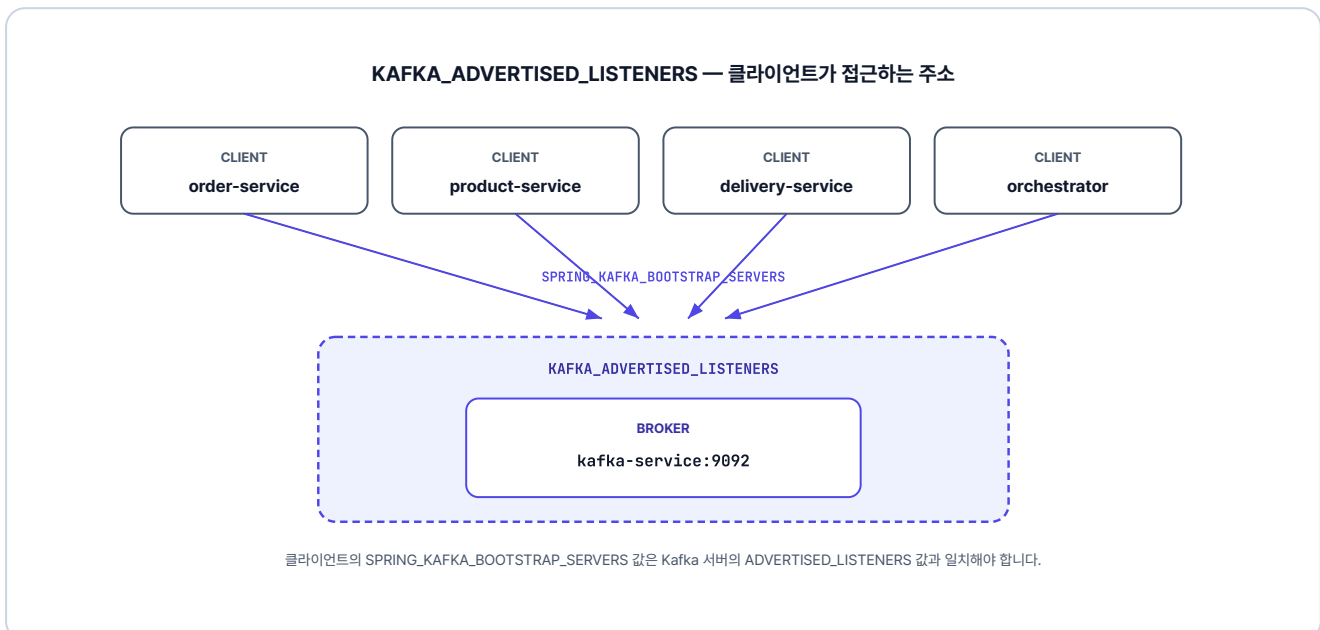


그림 4-14. 클라이언트는 kafka-service:9092로 Kafka에 접근합니다

`kafka-deploy.yml`의 핵심 환경변수는 다음과 같습니다. 전체 YAML은 GitHub에서 확인하세요.

환경변수	역할
<code>CLUSTER_ID</code>	Kafka 서버들을 하나의 클러스터로 묶는 식별자입니다.
<code>KAFKA_PROCESS_ROLES</code>	한 컨테이너가 브로커와 컨트롤러 역할을 겸하도록 지정합니다.
<code>KAFKA_ADVERTISED_LISTENERS</code>	클라이언트가 접근할 주소를 알립니다. ConfigMap의 <code>SPRING_KAFKA_BOOTSTRAP_SERVERS</code> 와 같아야 합니다.
<code>KAFKA_CONTROLLER_QUORUM_VOTERS</code>	컨트롤러 역할을 맡는 Kafka 서버 목록을 지정합니다.

4.6 실행 및 결과 확인

4.6.1 이미지 빌드

Minikube 내부에 이미지를 빌드합니다. 챕터 3 대비 orchestrator 서비스가 새로 추가됩니다.

터미널 이미지 빌드

```
minikube image build -t metacoding/db:2 ./db
minikube image build -t metacoding/gateway:2 ./gateway
minikube image build -t metacoding/order:2 ./order
minikube image build -t metacoding/product:2 ./product
minikube image build -t metacoding/user:2 ./user
minikube image build -t metacoding/delivery:2 ./delivery
minikube image build -t metacoding/orchestrator:2 ./orchestrator
```

4.6.2 배포 순서

Kafka가 준비되기 전에 서비스가 시작되면 연결 오류가 발생합니다. Kafka를 먼저 배포하고 ready 상태를 확인한 다음 나머지를 배포합니다.

터미널 배포 순서 (Kafka 우선)

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. Kafka 먼저 배포
kubectl apply -f k8s/kafka

# 3. Kafka가 준비될 때까지 대기
kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

# 4. 나머지 서비스 배포
kubectl apply -f k8s/db
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/gateway
kubectl apply -f k8s/orchestrator

# 5. Ingress 활성화 (최초 1회)
minikube addons enable ingress
```



kubectl apply · Kafka + 서비스 배포

```
namespace/metacoding created

[1] Kafka 우선 배포
deployment.apps/kafka-deploy created
service/kafka-service created
pod/kafka-xxx condition met (28s)

[2] 나머지 서비스 배포
db · order · product · user · delivery · gateway · orchestrator ...

-----
8개 Deployment + 7개 Service 배포 완료 █
```

그림 4-15. Kafka 및 서비스 배포 실행

모든 Pod가 Running 상태인지 확인합니다.

터미널 Pod 상태 확인

```
kubectl get pods -n metacoding
```

```
kubectl get pods -n metacoding
```

NAME	READY	STATUS	AGE
kafka-deploy-7d4c8b9f5-2xk9p	1/1	Running	2m
db-deploy-6f9b7c4d8-m4t2q	1/1	Running	90s
gateway-deploy-5c8d6f7b9-h7w3r	1/1	Running	88s
order-deploy-8b7f6c9d4-q2k8m	1/1	Running	85s
product-deploy-7c9d8b6f5-x4r2t	1/1	Running	83s
user-deploy-6d8c7b9f4-p3m9k	1/1	Running	80s
delivery-deploy-9f7c8b6d5-t6w2x	1/1	Running	78s
orchestrator-deploy-8c6f9b7d4-k9m4q	1/1	Running	75s

8개 Pod Running (Kafka + orchestrator 추가)

그림 4-16. Pod 상태 확인 (kubectl get pods)

4.6.3 서비스 접근

Ingress를 통해 외부에서 접속하려면 `minikube tunnel` 을 실행합니다.

```
터미널 외부 접근 터널

minikube tunnel
```

터널이 실행되면 `http://127.0.0.1:80` 로 gateway-service에 접속할 수 있습니다.

4.6.4 비동기 흐름 테스트

AirPods (productId=3)를 주문합니다.

```
POST http://127.0.0.1:80/api/orders

{
  "productId": 3,
  "quantity": 2,
  "price": 300000,
  "address": "Addr 4"
}
```



그림 4-17. 주문 생성 응답 (PENDING 상태)

챗터 3과 다르게 즉시 **PENDING** 상태로 반환됩니다. Kafka 이벤트가 처리되면 상태가 **COMPLETED** 로 변경됩니다. 잠시 후 주문 상태를 다시 조회합니다.

GET <http://127.0.0.1:80/api/orders/4>



그림 4-18. 주문 완료 확인 (COMPLETED 상태)

오픈이 : "PENDING이었다가 COMPLETED로 바뀌었어요! 그런데 롤백도 자동으로 되나요?"

선배 : "품질 상품으로 주문해 봐요. orchestrator 로그를 보면 보일 거예요."

4.6.5 롤백 확인 - 품질 상품 주문

동기 방식에서는 품질 상품이 첫 단계에서 막혀 되돌릴 것이 없었습니다. 반면 비동기 방식에서는 주문이 PENDING으로 먼저 저장됩니다. 그래서 재고 감소가 실패하면 orchestrator가 그 주문을 취소하는 롤백을 시작합니다. iPhone 15(productId=2, 재고 0)로 확인합니다.

POST <http://127.0.0.1:80/api/orders>

```
{
  "productId": 2,
  "quantity": 1,
  "price": 1300000,
  "address": "Addr 5"
}
```

상태: 200 • OK 시간: 30 ms 크기: 351 B

JSON Raw 헤더 4 테스트 결과

응답 본문

```
1  {
2    "status": 200,
3    "msg": "성공",
4    "body": {
5      "id": 5,
6      "userId": 1,
7      "productId": 2,
8      "quantity": 1,
9      "price": 1300000,
10     "status": "PENDING",
11     "createdAt": "2026-05-19T07:03:48.603301791",
12     "updatedAt": "2026-05-19T07:03:48.60336786"
13   }
14 }
```

그림 4-19. 품질 상품 주문 요청

잠시 후 상태를 확인하면 **CANCELLED** 가 됩니다.

GET <http://127.0.0.1:80/api/orders/5>



그림 4-20. 주문 취소 확인 (CANCELLED 상태)

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

터미널 리소스 정리

```
kubectl delete namespace metacoding
```

상품·배달을 직접 호출하고 실패까지 떠안던 주문 서비스는 이제 주문을 저장하고 이벤트 하나만 발행합니다. 정상 주문은 곧바로 PENDING으로 응답한 뒤 비동기로 COMPLETED가 되고, 품질 주문은 orchestrator가 알아서 CANCELLED로 되돌립니다. 직접 호출이 사라지면서 서비스 사이의 강한 결합이 풀렸고, try/catch 보상 블록도 함께 없어졌습니다.

며칠 뒤, 같은 동료가 책상 앞에 다시 섰습니다. 표정에서 불만이 묻어났습니다.

동료 : "어제 직접 주문해 봤는데, 좀 이상한 게 있었어요."

이것만은 기억하자

- 동기 REST 호출을 **Kafka 비동기 이벤트**로 교체하여 서비스 간 결합을 끊었습니다.
- **orchestrator** 서비스가 Kafka 토픽을 통해 전체 워크플로우를 조율합니다.
- `ConcurrentHashMap`으로 주문별 진행 상태를 추적하고, 실패 시 이미 처리된 단계만 자동 롤백합니다.
- order-service는 주문 생성 즉시 `PENDING` 상태로 반환하고, 처리 완료 후 `COMPLETED`로 갱신됩니다.

다음 챕터에서는 이를 해결합니다. 배달 기사가 완료 API를 호출하여 실제 배달 완료를 처리하고, WebSocket으로 클라이언트에게 주문 완료를 실시간 Push로 알립니다.